



PEGAUS HUGGING FACE TRANSFORMERS

LLMS FOR NLP & TEXT SUMMARIZATION





WHAT ARE LARGE LANGUAGE MODELS?

According to Stanford University, LLMs are advanced AI systems designed to understand, generate, and process human-like text by analyzing vast datasets using deep learning techniques, specifically transformer architectures





WHAT IS TEXT SUMMARIZATION?

Text summarization in NLP is the automated process of shortening long documents into concise versions while preserving key information and core meaning





PROJECT: RESUME SUMMARIZER

In this project, we use Pegasus transformer as the model base to generate text and summarize resume data.

Objectives:

- LLM / Transformer-based summarization
- Using Pegasus (Hugging Face Transformers)
- Applying it to resume summarization





PROJECT: RESUME SUMMARIZER



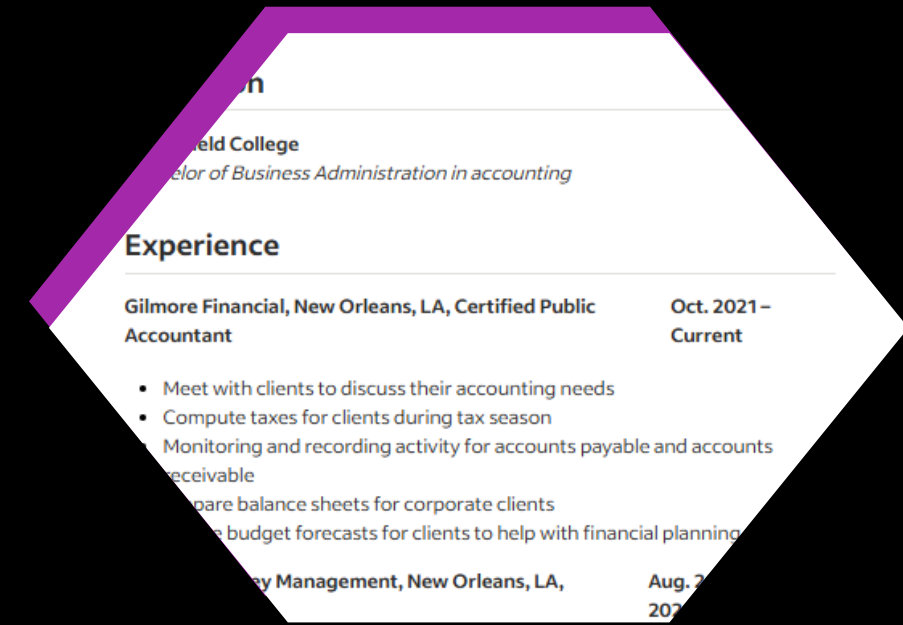
PEGASUS BY HUGGING FACE

- Transformer-based model
- Designed for abstractive summarization
- Trained on datasets like XSum



VS CODE

- An open-source code editor developed by Microsoft
- Supports Python programming language



DATA

- Test resume was sourced from Indeed



IMPORT LIBRARIES

```
[4] ✓ 21.6s Python
--- c:\Users\octav\anaconda3\envs\forhomework\Lib\site-packages\tqdm\auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://
    from .autonotebook import tqdm as notebook_tqdm

import sys
import torch
import transformers

print("Python:", sys.executable)
print("Torch version:", torch.__version__)
print("Transformers version:", transformers.__version__)

[5] ✓ 0.0s Python
--- Python: c:\Users\octav\anaconda3\envs\forhomework\python.exe
    Torch version: 2.11.0+cpu
    Transformers version: 5.5.0
```

The libraries used in this project are Pytorch and Pegasus tokenizer. Together to perform Natural Language Processing (NLP), specifically Abstractive Text Summarization.



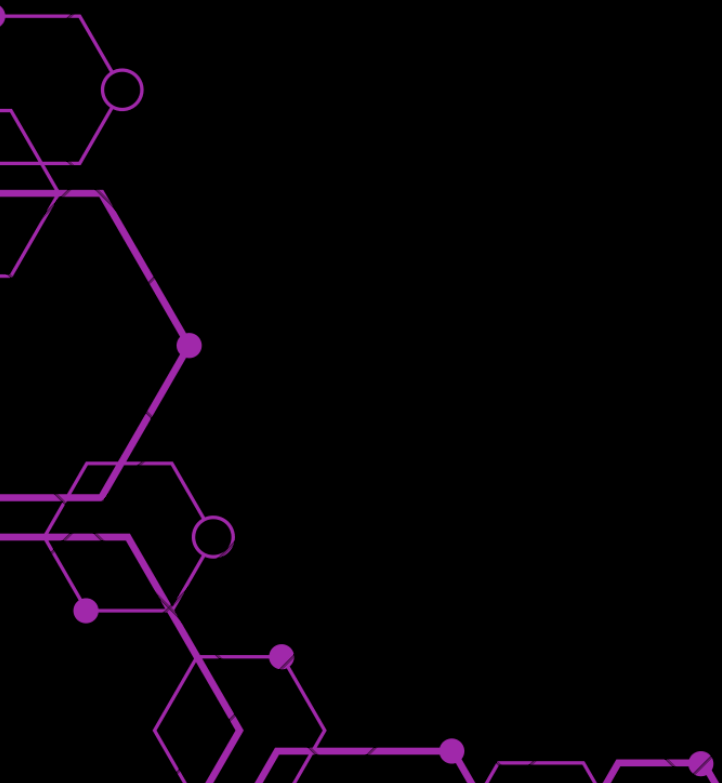
CREATE TOKENIZER

```
#create tokenizer and model
tokenizer = PegasusTokenizer.from_pretrained("google/pegasus-xsum")
```

[4] ✓ 2.0s Python

--- Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

This block initializes a tokenizer from the Hugging Face library specifically designed for the PEGASUS-XSum model. It downloads the necessary vocabulary and rules to convert raw text into numerical data that the model can process for concise summarization.





CREATE MODEL

```
#load model
model=PegasusForConditionalGeneration.from_pretrained("google/pegasus-xsum")
```

[7] ✓ 27.7s Python

--- Loading weights: 100%|██████████| 680/680 [00:00<00:00, 2028.14it/s]
PegasusForConditionalGeneration LOAD REPORT from: google/pegasus-xsum

Key	Status
model.decoder.embed_positions.weight	MISSING
model.encoder.embed_positions.weight	MISSING

Notes:
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

This line of code downloads the model's architectural weights (roughly 2.2GB) so it can generate concise, one-sentence summaries from your input text.



RESUME DATA

```
resume1 = """ Name: Jessica Langston
Seattle, WA | 206-555-0149 | jlang@email.com
Summary

Determined Accountant with three years of experience in personal and tax accounting. Excellent skills in mathematics, communication and organization with great attention to detail.

Education

Cloverfield College
Bachelor of Business Administration in accounting

Experience

Gilmore Financial, New Orleans, LA, Certified Public Accountant
Oct. 2021 - Current
Meet with clients to discuss their accounting needs
Compute taxes for clients during tax season
Monitoring and recording activity for accounts payable and accounts receivable
Prepare balance sheets for corporate clients
Create budget forecasts for clients to help with financial planning
Harrison Money Management, New Orleans, LA, Accountant
Aug. 2020 - Oct. 2021
Prepared tax returns for clients during tax season
Helped the internal audit team complete annual audits
Updated monthly reports to reflect the financial standings of each client
Harrison Money Management, New Orleans, LA, Junior Accountant
May 2019 - Aug. 2020
Maintained accounts receivable and accounts payable for the department
"""
```

Assign the data to a variable for future tokenization.



TOKENIZE DOCUMENT

```
# convert doc to tokens with minial parameters
tokens = tokenizer(resume1, truncation=True, padding="longest", return_tensors="pt")
print(tokens)
```

[9] ✓ 0.1s

Python

```
... {'input_ids': tensor([[ 5637,   151, 10288, 88515,  5060,   108,  7094, 1426, 27963,   121,
 38293, 14270, 13190, 1426,  7174, 19787, 1384, 11337,   107,   363,
15567, 60673, 31954,   122,   339,   231,   113,   306,   115,   510,
   111,  1035,  4261,   107, 10014,   766,   115, 10045,   108, 1660,
   111,  1134,   122,  255, 16587,   115,   623,  4261,   680,   107,
1988, 29849,  3483, 1369,  7058,   113, 1423,  4396,   115,  4261,
 5308, 40719,  3650,   108,   351,  7824,   108,  4546,   108,  5325,
 2397, 31954,  5177,   107, 30013,   212,  8708,  9196,   122,   658,
   112,  1693,   153,  4261,   397, 53176,  3467,   118,   658,   333,
 1035,   578, 16061,   111,  3429, 1383,   118,  2452, 11587,   111,
 2452, 48826, 12363, 1716,  4651,   118, 1933,   658,  5855, 1349,
13378,   118,   658,   112,   225,   122,   748, 1062, 11534,  5543,
 1764,   108,   351,  7824,   108,  4546,   108, 31954,  6262,   107,
 7149,   212,  5177,   107, 30013, 56882, 1035,  3115,   118,   658,
   333, 1035,   578,  4596,   316,   109,  2285,  7000,   320,   573,
1450, 17596, 16450,  2357, 1574,   112,  3291,   109,   748, 24111,
   113,   276, 1175, 11534,  5543, 1764,   108,   351,  7824,   108,
  4546,   108,  7273, 31954,   913, 1231,   212,  6262,   107,  7149,
23184,   316,  2452, 48826,   111,  2452, 11587,   118,   109,  2174,
50677,  2357, 10279,   112,   615,   126,   131,   116,  2598,   269,
 2807, 56882,   748, 1574,   112,   537,   122,   658,  9082,   116,
```

This line converts the resume1 text into numerical PyTorch tensors so the model can process it. It ensures the data fits the model's limits by truncating long text and padding shorter text to a uniform length.



FIRST MINIMAL CHANGES ON ATTEMPT: PARAMETER

```
[11] #summarize with minimal parameters
summary = model.generate(**tokens)
✓ 19.3s

[12] **tokens
✓ 0.0s

... {'input_ids': tensor([[ 5637,  151, 10288, 88515, 5060,  108, 7094, 1426, 27963,  121,
 38293, 14270, 13190, 1426, 7174, 19787, 1384, 11337,  107,  363,
15567, 60673, 31954,  122,  339,  231,  113,  306,  115,  510,
  111, 1035, 4261,  107, 10014,  766,  115, 10045,  108, 1660,
  111, 1134,  122,  255, 16587,  115,  623, 4261,  680,  107,
1988, 29849, 3483, 1369, 7058,  113, 1423, 4396,  115, 4261,
5308, 40719, 3650,  108,  351, 7824,  108, 4546,  108, 5325,
2397, 31954, 5177,  107, 30013,  212, 8708, 9196,  122,  658,
  112, 1693,  153, 4261,  397, 53176, 3467,  118,  658,  333,
1035,  578, 16061,  111, 3429, 1383,  118, 2452, 11587,  111,
2452, 48826, 12363, 1716, 4651,  118, 1933,  658, 5855, 1349,
13378,  118,  658,  112,  225,  122,  748, 1062, 11534, 5543,
1764,  108,  351, 7824,  108, 4546,  108, 31954, 6262,  107,
7149,  212, 5177,  107, 30013, 56882, 1035, 3115,  118,  658,
  333, 1035,  578, 4596,  316,  109, 2285, 7000,  320,  573,
```

```
[13] #summary in tokens
summary
✓ 0.0s

... tensor([[ 0, 49248,  231,  113,  306,  115,  510,  111, 1035, 4261,
 3881,    1]])

[14] #resume summary
tokenizer.decode(summary[0])
✓ 0.0s

... '<pad> 3+ years of experience in personal and tax accounting preferred</s>'
```

This line tells the model to process the tokens and generate a new sequence of text based on the input. It uses the model's learned patterns to produce the final abstractive summary as a set of numerical predicted IDs. Then converts the model's numerical output back into a human-readable sentence by reversing the tokenization process.



SECOND MODERATE CHANGES ATTEMPT: ON PARAMETERS

```
#summarize
summary2 = model.generate(tokens, max_length=80,
                           min_length=20,
                           num_beams=5,
                           length_penalty=1.2)
```

[34] ✓ 132s

Python

```
tokenizer.batch_decode(summary2[0], skip_special_tokens=True)
```

[34] ✓ 0.0s

Python

```
['Job Description: A Certified Public Accountant (CPA) with three years of experience in personal and tax accounting is looking for a new job in Seattle.']
```

This block of code generates a summary while using fine-tuning parameters like max_length and num_beams to control the output's size and quality. By adjusting these settings, you ensure the model produces a more structured and coherent summary instead of a simple, short prediction

Output: ['Job Description: A Certified Public Accountant (CPA) with three years of experience in personal and tax accounting is looking for a new job in Seattle.']



THIRD MODERATE CHANGES ATTEMPT: ON PARAMETERS

```
#summarize
summary3 = model.generate(**tokens, max_length=120,
                           min_length=20,
                           num_beams=5,
                           length_penalty=1.5,
                           early_stopping=True,)

[10] ✓ 17.2s Python
```

```
tokenizer.batch_decode(summary3[0], skip_special_tokens=True)

[11] ✓ 0.0s Python
```

```
... ['Job Description: A Certified Public Accountant (CPA) with three years of experience in personal and tax accounting is looking for a new job in Seattle, Washing!
```

This block allows for a longer output by increasing the `max_length` and `length_penalty`, which encourages the model to generate more detailed summaries. It also adds `early_stopping=True`, telling the model to stop generating as soon as all beam hypotheses reach a finished state to save time.

Output: ['Job Description: A Certified Public Accountant (CPA) with three years of experience in personal and tax accounting is looking for a new job in Seattle, Washington.']



FOURTH HEAVY CHANGES ON ATTEMPT: PARAMETERS

```
#changing parameters to get better summarization
summary4 = model.generate(tokens, max_length=200,
    min_length=40,
    num_beams=6,
    length_penalty=1.2,
    early_stopping=True,
    no_repeat_ngram_size=3)
```

[20] ✓ 27.9s

Python

```
tokenizer.batch_decode(summary4[0], skip_special_tokens=True)
```

[21] ✓ 0.0s

Python

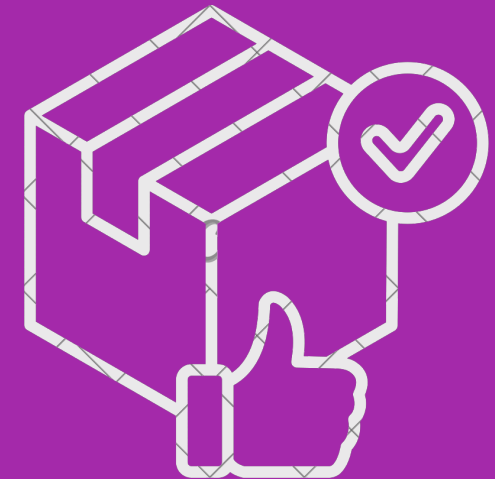
```
--- ['Reporting to the Chief Financial Officer, the Accountant is responsible for the day-to-day operations of the accounting department, including, but not limited to, preparing tax returns, recording activity for accounts payable and accounts receivable, and reviewing financial reports to share with clients.']
```

This version increases the summary's potential detail by raising the length limits and search quality (`num_beams=6`). It also adds `no_repeat_ngram_size=3` to specifically prevent the model from repeating the same three-word phrases, making the output sound more natural.

Output: ['Reporting to the Chief Financial Officer, the Accountant is responsible for the day-to-day operations of the accounting department, including, but not limited to, preparing tax returns, recording activity for accounts payable and accounts receivable, and reviewing financial reports to share with clients.']



BEST USE CASES



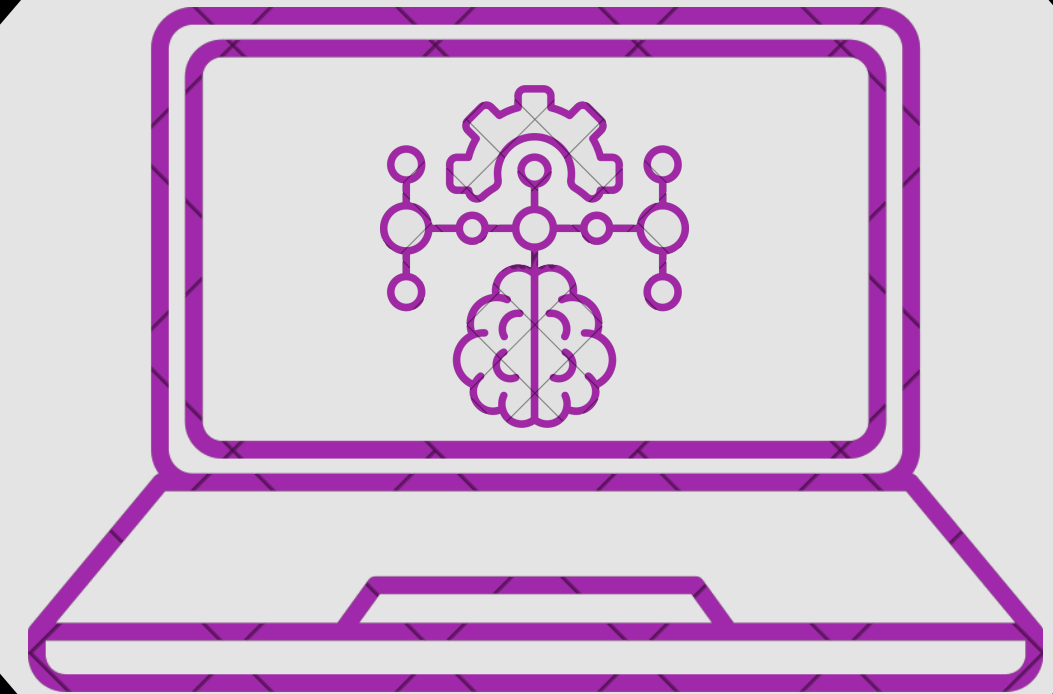
Summarizations for:

- News articles
- Reports
- Meeting & conversations
- Product Reviews
- Content Condensation



CONCLUSION

In conclusion, the best model was summary where it had a more accurate and coherent/cohesive summary of Jessica's work experience and responsibilities. This was done by adding parameters and adjusting their variables to get a better reading. To further elaborate, raising the length limits and search quality (num_beams=6) as well as adding no_repeat_ngram_size=3 to specifically prevent the model from repeating the same three-word phrases, the model's output became more coherent and precise. It is apparent that changing the minimum and maximum length as well as adding the no_repeat impacted the summary in a positive way.





THANK YOU

FOR ANY DATA PROJECTS CONNECT WITH ME ON LINKEDIN

